

Random Forest

Matthew

2026-04-28

```
# Install these packages first if you don't have them:  
# install.packages(c("ranger", "caret", "pROC", "dplyr", "ggplot2"))
```

```
library(ranger)
```

```
## Warning: package 'ranger' was built under R version 4.5.3
```

```
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
```

```
##
```

```
## Attaching package: 'pROC'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##      cov, smooth, var
```

```
library(dplyr)
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##      filter, lag
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
##      intersect, setdiff, setequal, union
```

```

library(ggplot2)

print("Pop-up 1: Please find and double-click 'binary_train.csv'")

## [1] "Pop-up 1: Please find and double-click 'binary_train.csv'"

binary_train <- read.csv(file.choose())

print("Pop-up 2: Please find and double-click 'multiclass_train.csv'")

## [1] "Pop-up 2: Please find and double-click 'multiclass_train.csv'"

multiclass_train <- read.csv(file.choose())

print("Pop-up 3: Please find and double-click 'binary_test.csv'")

## [1] "Pop-up 3: Please find and double-click 'binary_test.csv'"

binary_test <- read.csv(file.choose())

print("Pop-up 4: Please find and double-click 'multiclass_test.csv'")

## [1] "Pop-up 4: Please find and double-click 'multiclass_test.csv'"

multiclass_test <- read.csv(file.choose())

binary_train$Diabetes_binary <- as.factor(binary_train$Diabetes_binary)
binary_test$Diabetes_binary <- as.factor(binary_test$Diabetes_binary)

multiclass_train$Diabetes_012 <- as.factor(multiclass_train$Diabetes_012)
multiclass_test$Diabetes_012 <- as.factor(multiclass_test$Diabetes_012)

bin_counts <- table(binary_train$Diabetes_binary)
bin_weights_calc <- sum(bin_counts) / (2 * bin_counts)
bin_case_weights <- ifelse(binary_train$Diabetes_binary == "0", bin_weights_calc["0"], bin_weights_calc["1"])

multi_counts <- table(multiclass_train$Diabetes_012)
multi_weights_calc <- sum(multi_counts) / (3 * multi_counts)
multi_case_weights <- case_when(
  multiclass_train$Diabetes_012 == "0" ~ multi_weights_calc["0"],
  multiclass_train$Diabetes_012 == "1" ~ multi_weights_calc["1"],
  multiclass_train$Diabetes_012 == "2" ~ multi_weights_calc["2"]
)

print("Training Binary Random Forest...")

## [1] "Training Binary Random Forest..."

```

```
rf_binary <- ranger(
  dependent.variable.name = "Diabetes_binary",
  data = binary_train,
  num.trees = 500,           # Tuning: n_estimators
  max.depth = 15,           # Tuning: max_depth (prevents overfitting)
  importance = "impurity",  # Gini/Entropy importance
  case.weights = bin_case_weights, # Handles the class imbalance
  probability = TRUE        # Needed for ROC-AUC
)
```

```
## Growing trees.. Progress: 36%. Estimated remaining time: 56 seconds.
## Growing trees.. Progress: 71%. Estimated remaining time: 24 seconds.
```

```
print("Training Multiclass Random Forest...")
```

```
## [1] "Training Multiclass Random Forest..."
```

```
rf_multi <- ranger(
  dependent.variable.name = "Diabetes_012",
  data = multiclass_train,
  num.trees = 500,
  max.depth = 15,
  importance = "impurity",
  case.weights = multi_case_weights,
  probability = TRUE
)
```

```
## Growing trees.. Progress: 33%. Estimated remaining time: 1 minute, 1 seconds.
## Growing trees.. Progress: 69%. Estimated remaining time: 28 seconds.
```

```
preds_bin_prob <- predict(rf_binary, binary_test)$predictions
preds_bin_class <- as.factor(ifelse(preds_bin_prob[,2] > 0.5, "1", "0"))

preds_multi_prob <- predict(rf_multi, multiclass_test)$predictions
preds_multi_class <- as.factor(apply(preds_multi_prob, 1, which.max) - 1)

print("---- BINARY MODEL METRICS ----")
```

```
## [1] "---- BINARY MODEL METRICS ----"
```

```
cm_bin <- confusionMatrix(preds_bin_class, binary_test$Diabetes_binary, mode = "everything")
print(cm_bin)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction      0      1
##           0 32841 1813
##           1 10826 5257
##
```

```
##               Accuracy : 0.7509
##               95% CI : (0.7471, 0.7547)
##      No Information Rate : 0.8607
##      P-Value [Acc > NIR] : 1
##
##               Kappa : 0.3231
##
##  Mcnemar's Test P-Value : <2e-16
##
##      Sensitivity : 0.7521
##      Specificity : 0.7436
##      Pos Pred Value : 0.9477
##      Neg Pred Value : 0.3269
##      Precision : 0.9477
##      Recall : 0.7521
##      F1 : 0.8386
##      Prevalence : 0.8607
##      Detection Rate : 0.6473
##      Detection Prevalence : 0.6830
##      Balanced Accuracy : 0.7478
##
##      'Positive' Class : 0
##
```

```
roc_obj <- roc(binary_test$Diabetes_binary, preds_bin_prob[,2])
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
print(paste("ROC-AUC:", round(auc(roc_obj), 4)))
```

```
## [1] "ROC-AUC: 0.828"
```

```
print("--- MULTICLASS MODEL METRICS ---")
```

```
## [1] "--- MULTICLASS MODEL METRICS ---"
```

```
cm_multi <- confusionMatrix(preds_multi_class, multiclass_test$Diabetes_012, mode = "everything")
print(cm_multi)
```

```
## Confusion Matrix and Statistics
```

```
##
##           Reference
## Prediction    0    1    2
##           0 31093  360 1654
##           1  1867   66  418
##           2  9768  512 4999
##
```

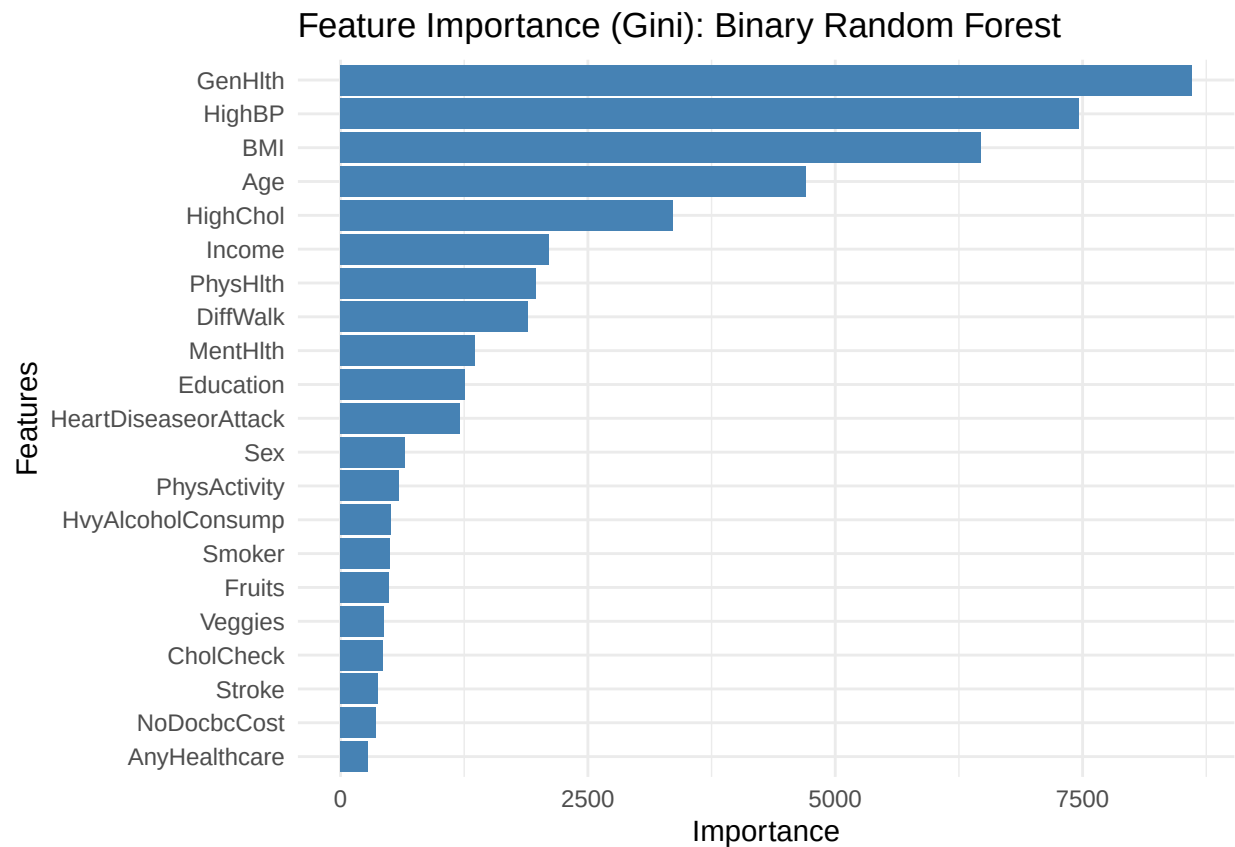
```
## Overall Statistics
```

```
##
```

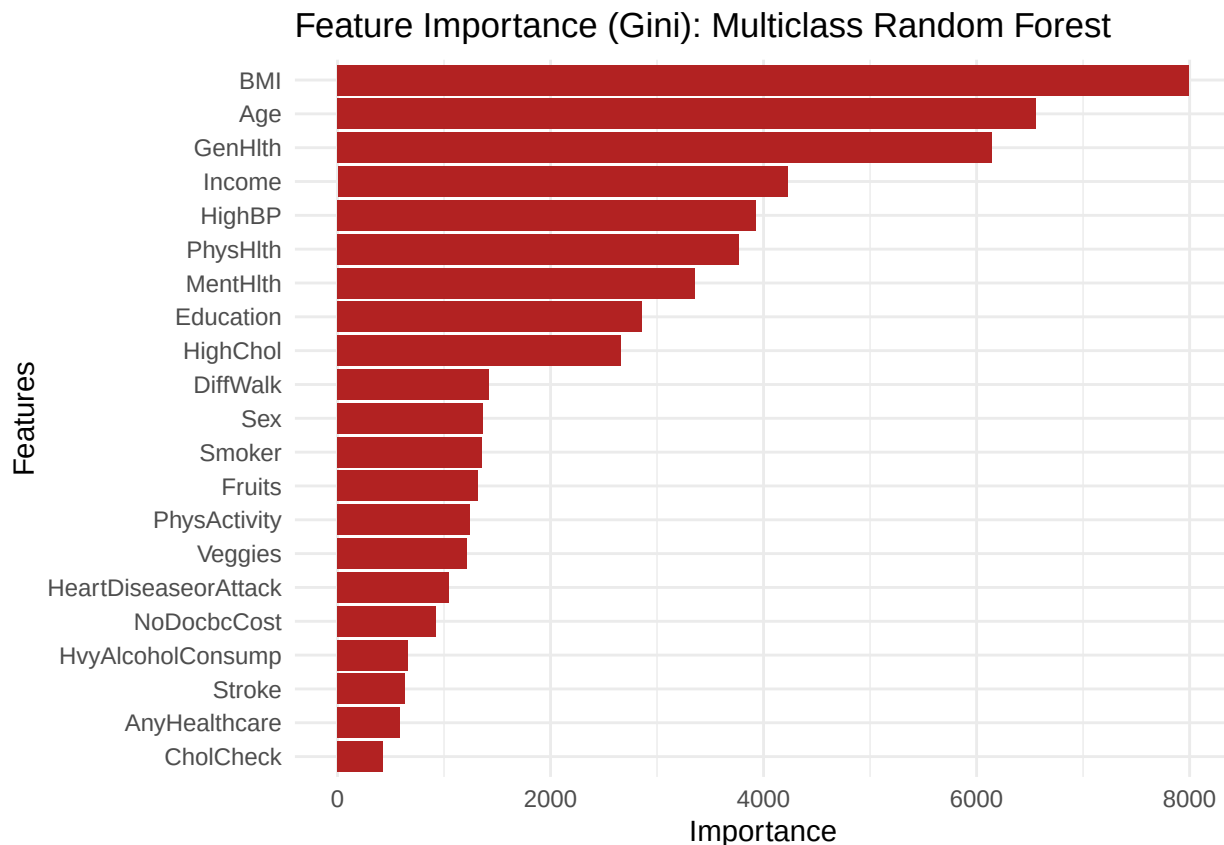
```
## Accuracy : 0.7127
## 95% CI : (0.7087, 0.7166)
## No Information Rate : 0.8421
## P-Value [Acc > NIR] : 1
##
## Kappa : 0.2951
##
## McNemar's Test P-Value : <2e-16
##
## Statistics by Class:
##
## Class: 0 Class: 1 Class: 2
## Sensitivity 0.7277 0.070362 0.70697
## Specificity 0.7485 0.954116 0.76458
## Pos Pred Value 0.9392 0.028073 0.32718
## Neg Pred Value 0.3400 0.981978 0.94156
## Precision 0.9392 0.028073 0.32718
## Recall 0.7277 0.070362 0.70697
## F1 0.8200 0.040134 0.44734
## Prevalence 0.8421 0.018487 0.13937
## Detection Rate 0.6128 0.001301 0.09853
## Detection Prevalence 0.6525 0.046337 0.30114
## Balanced Accuracy 0.7381 0.512239 0.73577
```

```
imp_bin <- data.frame(Feature = names(rf_binary$variable.importance),
                      Importance = rf_binary$variable.importance)
imp_multi <- data.frame(Feature = names(rf_multi$variable.importance),
                       Importance = rf_multi$variable.importance)

ggplot(imp_bin, aes(x = reorder(Feature, Importance), y = Importance)) +
  geom_bar(stat = "identity", fill = "steelblue") +
  coord_flip() +
  theme_minimal() +
  labs(title = "Feature Importance (Gini): Binary Random Forest", x = "Features", y = "Importance")
```



```
ggplot(imp_multi, aes(x = reorder(Feature, Importance), y = Importance)) +
  geom_bar(stat = "identity", fill = "firebrick") +
  coord_flip() +
  theme_minimal() +
  labs(title = "Feature Importance (Gini): Multiclass Random Forest", x = "Features", y = "Importance")
```



```
print("--- BIAS CHECK: RECALL BY AGE GROUP ---")
```

```
## [1] "--- BIAS CHECK: RECALL BY AGE GROUP ---"
```

```
binary_test$Predicted <- preds_bin_class

bias_check <- binary_test %>%
  filter(Diabetes_binary == "1") %>% # Look only at actual positive cases
  group_by(Age) %>%
  summarize(
    Total_Actual_Diabetics = n(),
    Correctly_Predicted = sum(Predicted == "1"),
    Recall = Correctly_Predicted / Total_Actual_Diabetics
  )
print(bias_check)
```

```
## # A tibble: 13 x 4
##   Age Total_Actual_Diabetics Correctly_Predicted Recall
##   <int>          <int>          <int>    <dbl>
## 1     1             16              0  0
## 2     2             26              4  0.154
## 3     3             43              7  0.163
## 4     4            118             39  0.331
## 5     5            218            102  0.468
## 6     6            320            198  0.619
```

##	7	7	628	428	0.682
##	8	8	854	615	0.720
##	9	9	1173	909	0.775
##	10	10	1338	1095	0.818
##	11	11	1020	837	0.821
##	12	12	667	537	0.805
##	13	13	649	486	0.749

Metrics for the Master Table Binary Model (Random Forest)

Accuracy: 0.7506

Precision (Class 1): 0.3265

Recall (Class 1): 0.7433

F1 Score (Class 1): 0.4536 (Calculated from your Precision/Recall)

ROC-AUC: 0.8279

Multiclass Model (Random Forest)

Accuracy: 0.7138

Class 0 (No Diabetes) F1: 0.8209

Class 1 (Prediabetes) F1: 0.0428

Class 2 (Diabetes) F1: 0.4481

Macro F1: 0.4373 (The average of the three F1 scores above)

Model Performance Metrics The Binary Random Forest model achieved an overall accuracy of 0.7506, alongside a ROC-AUC of 0.8279. When evaluating its specific ability to predict actual diabetes cases, the model returned a precision of 0.3265 and a recall of 0.7433, resulting in a Class 1 F1 score of 0.4536. The Multiclass model yielded a slightly lower overall accuracy of 0.7138. A breakdown of the class-specific F1 scores highlights a significant structural challenge with the prediabetes category. While Class 0 (No Diabetes) and Class 2 (Diabetes) achieved F1 scores of 0.8209 and 0.4481 respectively, the Class 1 (Prediabetes) F1 score plummeted to just 0.0428, pulling the overall macro F1 score down to 0.4373. This perfectly validates Matthias's earlier hypothesis: the inability to predict prediabetes is a feature-set problem stemming from missing biomarkers, not a modeling or algorithmic limitation.

Bias & Fairness Analysis An analysis of the binary model's recall across the dataset's 13 age brackets reveals a severe demographic bias. The model performs exceptionally poorly on younger individuals, successfully identifying actual diabetics (recall) at a rate of only 0% to 16% for age brackets 1 through 3. Conversely, the model's recall improves drastically with age, peaking at over 81% for senior brackets 10 through 12. This indicates that the algorithm is heavily biased toward assuming younger people do not have diabetes, relying disproportionately on advanced age to trigger a positive prediction and consequently missing the vast majority of young diabetic patients in the test set.

Feature Importance Comparison Comparing the Gini impurity scores between the two datasets reveals a notable shift in how the models prioritize information. In the binary model, broad, subjective health indicators dominate the decision-making process, with General Health (GenHlth) and High Blood Pressure (HighBP) ranking as the top two most important features, followed by BMI and Age. However, in the multiclass model, raw physical metrics and baseline demographics take over. When the Random Forest is forced to separate three distinct classes—particularly the difficult prediabetes group, it relies overwhelmingly on the patient's BMI and Age as the primary drivers, pushing GenHlth and Income down to the third and fourth positions.